

MODULO 01

Clean Architecture para Flutter

Guia completa para aplicar Clean Architecture en Flutter con Supabase. Del codigo espagueti a una arquitectura escalable, testeable y mantenible.

ARQUITECTURA

4 Capas · Domain-Driven · SOLID

BACKEND

Supabase (BaaS) · REST API · Isar

NIVEL

Principiante a Intermedio

TIEMPO ESTIMADO

6-8 horas

Clean Architecture

Flutter

Supabase

SOLID

fpdart

Contenido

15 capitulos · 15 archivos · ~6-8 horas de lectura

PARTE 1 · FUNDAMENTOS

01 Guía Completa de Clean Architecture para Flutter

02

03

04

PARTE 2 · IMPLEMENTACION CRUD

05

05a

05b

05c

PARTE 3 · INFRAESTRUCTURA

06

07

08

PARTE 4 · MADUREZ Y CIERRE

09

10 10: SOLID Explícite con Dart/Flutter

11 11: Anti-patrones en Clean Architecture

12 12: Cuando NO Usar Clean Architecture

Guía Completa de Clean Architecture para Flutter

Documento unificado que combina Introducción, Filosofía, Estructura, Flujo de Datos, Implementación Práctica con Sistema de Usuarios CRUD, Inyección de Dependencias, Testing y Templates.

Este documento es parte de la sección 01 de la guía. Navega por los archivos numerados para seguir el orden de aprendizaje.

1. Introducción y Filosofía

¿Qué es Clean Architecture?

Clean Architecture, propuesta por Robert C. Martin (Uncle Bob), es un diseño de software que separa el código en capas independientes con una **regla de dependencia estricta**: las capas externas dependen de las internas, pero las internas no saben nada de las externas.

La Analogía del Restaurante

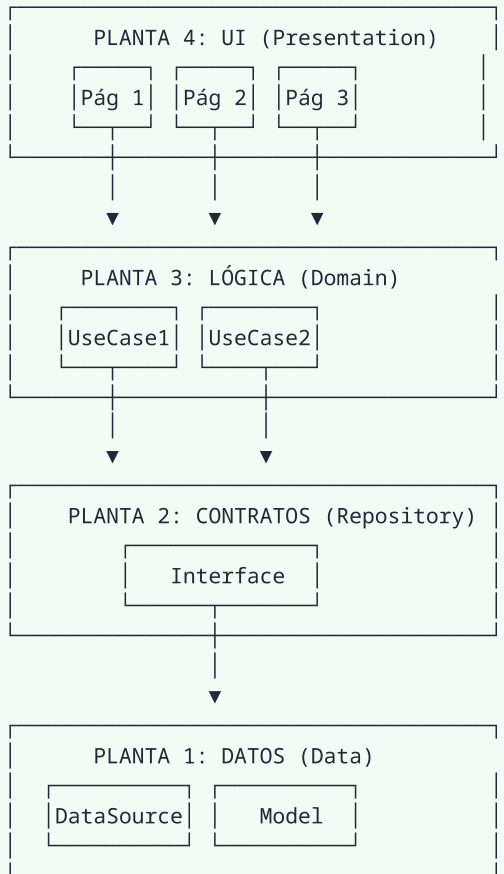
Imagina un restaurante donde pides una hamburguesa:

Tu Código	Restaurante	Qué Hace
Domain	La Receta	Dice qué ingredientes necesitas
Data	La Cocina	Cocina los ingredientes
Repository	El Almacén	Decide si usa carne fresca o congelada
UseCase	El Chef	Sigue la receta paso a paso
Cubit	El Mesero	Lleva tu pedido y trae la comida
UI	Tu Mesa	Donde comes y pides

Regla importante: Tú (UI) **NUNCA** entras a la cocina (Data). Todo pasa por el mesero (Cubit).

La Analogía del Edificio de Oficinas

Clean Architecture organiza el código como un edificio de varias plantas:



Regla de Oro: Código de plantas superiores NUNCA conoce detalles de plantas inferiores.

¿Por qué usar Clean Architecture?

- **Independencia del Framework:** El núcleo de tu lógica de negocio no depende de Flutter.
- **Testabilidad:** Cada capa se puede probar de forma aislada.
- **Escalabilidad y Mantenimiento:** Fácil añadir features o cambiar implementaciones.
- **Organización:** Código claramente organizado por funcionalidad y capa.

El Problema: Código Espagueti

Imagina un plato de espagueti donde todo está mezclado:

- UI con lógica de negocio
- Llamadas HTTP en los widgets
- Base de datos acoplada a la interfaz
- Imposible de testear
- Un cambio rompe todo

2. Las 4 Capas de Clean Architecture

Diagrama de las 4 Capas

```
graph TD; C4["CAPA 4 - UI (Widgets)"] --> C3["CAPA 3 - Presentation (Cubit)"]; C3 --> C2["CAPA 2 - Domain (UseCase + Entity)"]; C2 --> C1["CAPA 1 - Data (Repository + DataSource)"];
```

↓ Llama al Cubit

↓ Llama al UseCase

↓ Llama al Repository

↓ Habla con API o Base de Datos

Las 4 Capas en Detalle

1 Domain (El Núcleo)

Contiene: Entities, Repository Interfaces, Use Cases

Principios:

- Pura lógica de negocio
- Sin dependencias externas (no Flutter, no HTTP, no DB)
- Altamente testeable
- Reutilizable en otros proyectos

Analogía: Las reglas del juego de ajedrez

2 Data (La Implementación)

Contiene: Models, DataSources, Repository Implementations

Principios:

- Implementa los contratos del Domain
- Habla con APIs, bases de datos, cache
- Convierte datos externos a Entities

Analogía: El tablero físico y las piezas de ajedrez

3 Presentation (El Estado)

Contiene: Cubits/Blocs, States

Principios:

- Maneja el estado de la UI
- Orquesta Use Cases
- Sin lógica de negocio compleja

Analogía: El visor que muestra el tablero en tu celular

4 UI (La Vista)

Contiene: Widgets, Pages, Screens

Principios:

- Solo muestra datos

- Recibe eventos del usuario
- Se reconstruye cuando cambia el estado

Analogía: La pantalla de tu celular

Principios Fundamentales

- **Capas:** La arquitectura se divide en capas (Presentación, Dominio, Datos).
 - **Regla de Dependencia:** El código fuente solo puede depender "hacia adentro".
 - **Abstracciones:** Las capas se comunican a través de interfaces (clases abstractas en Dart).
-

3. Estructura de Carpetas

Estructura Base Universal

```

lib/
├── core/                                # Código compartido entre features
│   ├── common/                          # Clases base (UseCase, etc.)
│   │   └── usecase.dart
│   ├── data/                            # Configuración de datos global
│   │   └── local/
│   │       ├── dashboard_initializer.dart
│   │       └── isar_service.dart
│   ├── di/                              # Inyección de dependencias
│   │   ├── injection_container.dart
│   │   └── service_locator.dart
│   ├── error/                           # Manejo de errores
│   │   ├── exceptions.dart
│   │   └── failures.dart
│   ├── network/                         # Monitoreo de conectividad
│   │   └── network_info.dart
│   ├── routing/                         # Navegación
│   │   └── app_router.dart
│   ├── services/                       # Servicios globales (CacheManager, etc.)
│   │   └── cache_manager.dart
│   ├── session/                        # Sesión del usuario autenticado
│   │   ├── user_session.dart
│   │   └── user_session_impl.dart
│   ├── theme/                          # Tema y estilos
│   │   └── app_theme.dart
│   └── utils/                           # Utilidades
│       ├── constants.dart
│       └── widgets/                     # Widgets reutilizables
├── features/                            # Cada feature tiene su propia estructura
│   ├── {feature_name}/
│   │   ├── data/
│   │   │   ├── datasources/
│   │   │   │   ├── {feature}_local_data_source.dart
│   │   │   │   └── {feature}_remote_data_source.dart # Opcional
│   │   │   ├── models/
│   │   │   │   └── {feature}_model.dart
│   │   │   └── repositories/
│   │   │       └── {feature}_repository_impl.dart
│   │   ├── domain/
│   │   │   ├── entities/
│   │   │   │   └── {feature}.dart
│   │   │   ├── repositories/
│   │   │   │   └── {feature}_repository.dart
│   │   │   └── usecases/
│   │   │       ├── get_{feature}.dart
│   │   │       ├── create_{feature}.dart
│   │   │       └── delete_{feature}.dart
│   │   └── presentation/
│   │       ├── cubit/
│   │       │   ├── {feature}_cubit.dart
│   │       │   └── {feature}_state.dart
│   │       └── pages/
│   │           └── {feature}_page.dart
└── main.dart

```

Reglas de Organización

✓ Hacer:

- Una carpeta por feature
- Feature es independiente de otras features
- Core no depende de features
- Cada capa en su carpeta correspondiente

✗ No Hacer:

```
lib/
├─ data/           # ✗ Mal: Todos los datos juntos
├─ domain/         # ✗ Mal: Todas las entidades juntas
├─ ui/             # ✗ Mal: Todas las pantallas juntas
└─ models/         # ✗ Mal: Todos los modelos juntos
```

✓ Hacer:

```
lib/
├─ features/
│   ├── user/      # ✓ Bien: Todo lo de usuario aquí
│   ├── product/   # ✓ Bien: Todo lo de producto aquí
│   └─ order/      # ✓ Bien: Todo lo de orden aquí
└─ core/           # ✓ Bien: Solo código compartido
```


4. Flujo de Datos

Diagrama de Flujo Completo

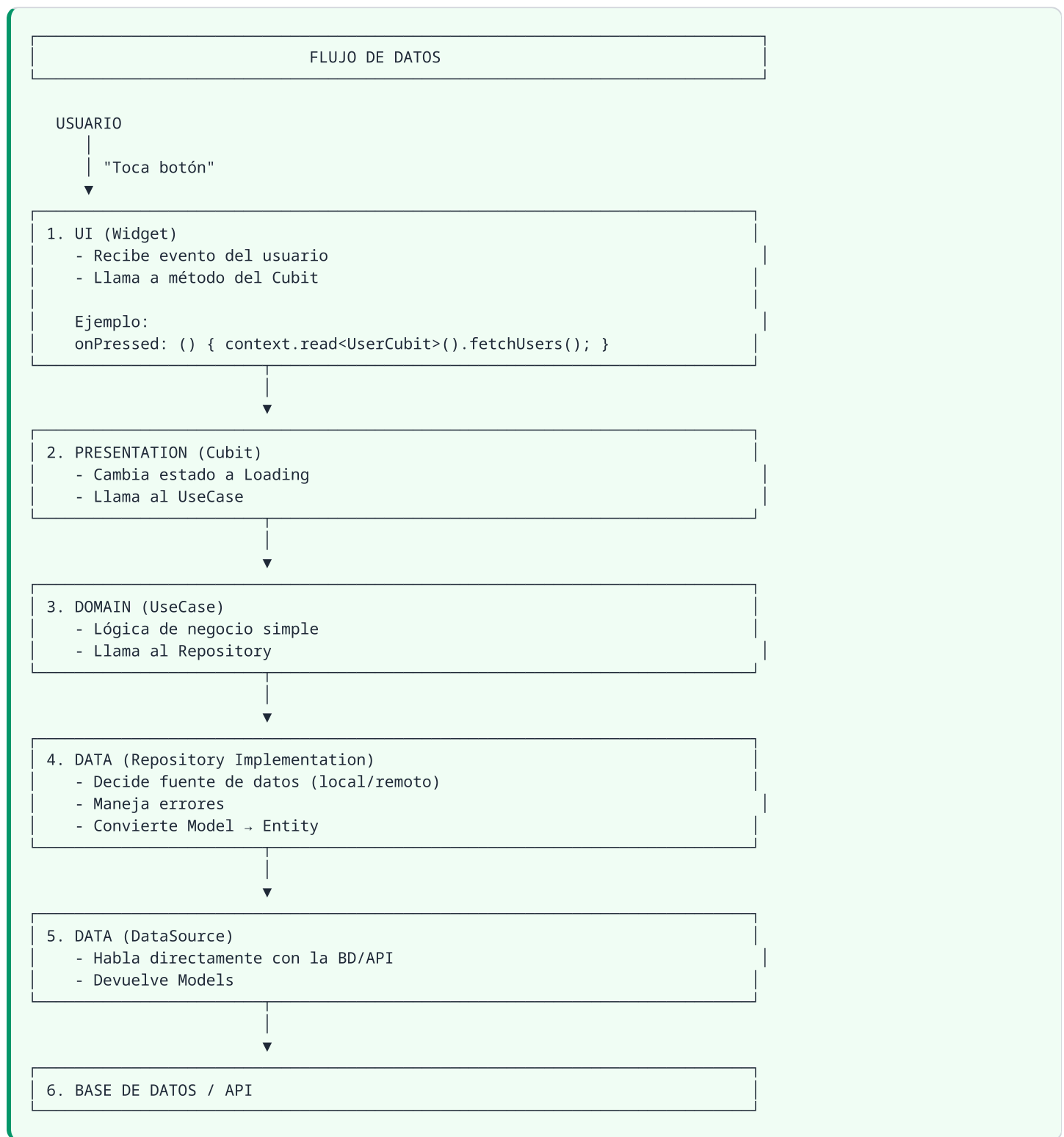


Diagrama Mermaid

```
graph TD
    %% Capa de Presentación
    A[Widget/UI] -->|Llama a función| B[Cubit]
    B -->|Emitiendo estado| C[State]
    C -->|Actualiza UI| A

    %% Cubit a Dominio
    B -->|Llama a use case| D[UseCase]
    D -->|Retorna Either<Failure, Entity>| B

    %% Dominio a Datos
    D -->|Llama a interfaz| E[Repository Interface]
    E -->|Retorna Either<Failure, Entity>| D

    %% Implementación de Repositorio
    F[Repository Implementation] -->|Implementa| E
    F -->|Llama a datasource| G[DataSource]
    G -->|Retorna Model| F
    F -->|Convierte Model a Entity| E

    %% Origen de datos externos
    G -->|Hace request| H[API/Database]
    H -->|Retorna datos crudos| G

    %% Estilos
    classDef presentation fill:#e1f5fe,stroke:#01579b,stroke-width:2px
    classDef domain fill:#f3e5f5,stroke:#4a148c,stroke-width:2px
    classDef data fill:#e8f5e8,stroke:#1b5e20,stroke-width:2px
    classDef external fill:#fff3e0,stroke:#e65100,stroke-width:2px

    class A,B,C presentation
    class D,E domain
    class F,G data
    class H external
```

Ejemplo Paso a Paso: Operación CRUD (Leer datos)

- UI (Widget):** Un botón es presionado. El `onPressed` llama a `context.read<UserCubit>().fetchUsers();`.
- Cubit (Presentation):** El `UserCubit` recibe la llamada. Emite un estado de carga: `emit(UserLoading())`. Luego, llama al `UseCase`.
- UseCase (Domain):** El `GetUsersUseCase` llama al método del `Repository`.
- Repository Impl (Data):** Decide cómo obtener los datos. Si hay internet, pide a la fuente remota. Si no, usa la caché local.
- DataSource (Data):** Usa `dio` o `isar` para obtener los datos. Devuelve una lista de `UserModel`.
- Retorno del Flujo:**
 - El `DataSource` devuelve `List<UserModel>` al `Repository Impl`.
 - El `Repository Impl` convierte `List<UserModel>` a `List<UserEntity>`.
 - El `UseCase` recibe `Either<Failure, List<UserEntity>>` y lo devuelve al `Cubit`.
 - El `Cubit` procesa el resultado:
 - Si es un `Failure`, emite `emit(UserError(message))`.
 - Si es éxito, emite `emit(UsersLoaded(users))`.
- UI (Widget):** Un `BlocBuilder` escucha los cambios de estado y reconstruye la UI.

Regla de Dependencia

Las flechas de dependencia SIEMPRE apuntan hacia adentro:

UI → Presentation → Domain → Data

× Esto está PROHIBIDO:

- Domain → UI (Domain NO puede saber de UI)
- Data → Domain implementación (Domain solo interfaces)
- UI → Data directo (Siempre pasar por Presentation y Domain)

¿Dónde van los Errores?

Tipo de Error	¿En qué capa?	¿Qué es?	Ejemplo
Exception	Data Layer	Error técnico (HTTP, BD, parsing)	ServerException, CacheException
Failure	Domain Layer	Error de negocio (entendible por usuario)	ServerFailure, NetworkFailure

Flujo de errores:

```
DataSource lanza Exception (error técnico)
  ↓
Repository atrapa Exception con try/catch
  ↓
Repository convierte Exception → Failure
  ↓
Repository retorna Either<Failure, T>
  ↓
UseCase recibe Failure
  ↓
Cubit emite estado de error
  ↓
UI muestra mensaje amigable al usuario
```

5. Implementación Práctica: Sistema de Usuarios CRUD

Vamos a implementar un sistema CRUD completo de usuarios.

Requerimientos

1. Ver lista de usuarios
2. Crear usuario
3. Ver detalle de usuario
4. Eliminar usuario

Estructura de Archivos

```
lib/features/user/  
├── data/  
│   ├── datasources/  
│   │   └── user_local_data_source.dart  
│   ├── models/  
│   │   ├── user_model.dart  
│   │   └── user_model.g.dart  
│   └── repositories/  
│       └── user_repository_impl.dart  
├── domain/  
│   ├── entities/  
│   │   └── user.dart  
│   ├── repositories/  
│   │   └── user_repository.dart  
│   └── usecases/  
│       ├── create_user.dart  
│       ├── delete_user.dart  
│       ├── get_user.dart  
│       └── get_users.dart  
└── presentation/  
    ├── cubit/  
    │   ├── user_cubit.dart  
    │   └── user_state.dart  
    └── pages/  
        ├── user_detail_page.dart  
        └── users_list_page.dart
```

5.1 Domain Layer (La Receta)

Entity

Archivo: `lib/features/user/domain/entities/user.dart`

```
import 'package:equatable/equatable.dart';

class User extends Equatable {
  const User({
    required this.id,
    required this.name,
    required this.email,
    this.isActive = true,
    this.createdAt,
    this.avatarUrl,
  });

  final String id;
  final String name;
  final String email;
  final bool isActive;
  final DateTime? createdAt;
  final String? avatarUrl;

  bool get hasAvatar => avatarUrl != null && avatarUrl!.isNotEmpty;

  bool get isNew {
    if (createdAt == null) return false;
    final daysSinceCreated = DateTime.now().difference(createdAt!).inDays;
    return daysSinceCreated < 7;
  }

  User copyWith({
    String? id,
    String? name,
    String? email,
    bool? isActive,
    DateTime? createdAt,
    String? avatarUrl,
  }) {
    return User(
      id: id ?? this.id,
      name: name ?? this.name,
      email: email ?? this.email,
      isActive: isActive ?? this.isActive,
      createdAt: createdAt ?? this.createdAt,
      avatarUrl: avatarUrl ?? this.avatarUrl,
    );
  }

  @override
  List<Object?> get props => [id, name, email, isActive, createdAt, avatarUrl];

  @override
  String toString() => 'User(id: $id, name: $name)';
}
```

Repository Interface

Archivo: `lib/features/user/domain/repositories/user_repository.dart`

```
import 'package:fpdart/fpdart.dart';
import 'package:my_app/core/error/failures.dart';
import 'package:my_app/features/user/domain/entities/user.dart';

abstract class UserRepository {
  Future<Either<Failure, List<User>>> getUsers();
  Future<Either<Failure, User>> getUser(String id);
  Future<Either<Failure, void>> createUser(User user);
  Future<Either<Failure, void>> updateUser(User user);
  Future<Either<Failure, void>> deleteUser(String id);
}
```

UseCases

Archivo: `lib/features/user/domain/usecases/get_users.dart`

```
import 'package:fpdart/fpdart.dart';
import 'package:my_app/core/common/usecase.dart';
import 'package:my_app/core/error/failures.dart';
import 'package:my_app/features/user/domain/entities/user.dart';
import 'package:my_app/features/user/domain/repositories/user_repository.dart';

class GetUsers extends UseCase<List<User>, NoParams> {
  final UserRepository repository;

  GetUsers(this.repository);

  @override
  Future<Either<Failure, List<User>>> call(NoParams params) async {
    return await repository.getUsers();
  }
}
```

Archivo: `lib/features/user/domain/usecases/get_user.dart`

```
import 'package:fpdart/fpdart.dart';
import 'package:equatable/equatable.dart';
import 'package:my_app/core/common/usecase.dart';
import 'package:my_app/core/error/failures.dart';
import 'package:my_app/features/user/domain/entities/user.dart';
import 'package:my_app/features/user/domain/repositories/user_repository.dart';

class GetUser extends UseCase<User, GetUserParams> {
  final UserRepository repository;

  GetUser(this.repository);

  @override
  Future<Either<Failure, User>> call(GetUserParams params) async {
    return await repository.getUser(params.id);
  }
}

class GetUserParams extends Equatable {
  final String id;

  const GetUserParams(this.id);

  @override
  List<Object?> get props => [id];
}
```

Archivo: `lib/features/user/domain/usecases/create_user.dart`

```

import 'package:fpdart/fpdart.dart';
import 'package:equatable/equatable.dart';
import 'package:my_app/core/common/usecase.dart';
import 'package:my_app/core/error/failures.dart';
import 'package:my_app/features/user/domain/entities/user.dart';
import 'package:my_app/features/user/domain/repositories/user_repository.dart';

class CreateUser extends UseCase<void, CreateUserParams> {
  final UserRepository repository;

  CreateUser(this.repository);

  @override
  Future<Either<Failure, void>> call(CreateUserParams params) async {
    final user = User(
      id: DateTime.now().millisecondsSinceEpoch.toString(),
      name: params.name,
      email: params.email,
      createdAt: DateTime.now(),
    );

    return await repository.createUser(user);
  }
}

class CreateUserParams extends Equatable {
  final String name;
  final String email;

  const CreateUserParams({required this.name, required this.email});

  @override
  List<Object?> get props => [name, email];
}

```

Archivo: [lib/features/user/domain/usecases/delete_user.dart](#)

```

import 'package:fpdart/fpdart.dart';
import 'package:equatable/equatable.dart';
import 'package:my_app/core/common/usecase.dart';
import 'package:my_app/core/error/failures.dart';
import 'package:my_app/features/user/domain/repositories/user_repository.dart';

class DeleteUser extends UseCase<void, DeleteUserParams> {
  final UserRepository repository;

  DeleteUser(this.repository);

  @override
  Future<Either<Failure, void>> call(DeleteUserParams params) async {
    return await repository.deleteUser(params.id);
  }
}

class DeleteUserParams extends Equatable {
  final String id;

  const DeleteUserParams(this.id);

  @override
  List<Object?> get props => [id];
}

```

5.1.1 Comparación: Future vs TaskEither

`fpdart` ofrece dos formas de manejar operaciones asíncronas con errores. Aquí te mostramos ambas para que elijas la que prefieras.

Opción A: `Future<Either>` (Más Familiar)

Esta es la forma tradicional, similar a lo que ya conoces. Funciona exactamente igual con `fpdart`.

```
import 'package:fpdart/fpdart.dart';

class GetUsers extends UseCase<List<User>, NoParams> {
  final UserRepository repository;

  GetUsers(this.repository);

  @override
  Future<Either<Failure, List<User>>> call(NoParams params) async {
    return await repository.getUsers();
  }
}
```

Uso en el Cubit:

```
final result = await _getUsers(NoParams());
result.match(
  (failure) => emit(UserError(failure.toString())),
  (users) => emit(UsersLoaded(users)),
);
```

Pros:

- ✓ Más familiar para quienes vienen de `dartz`
- ✓ Fácil de entender
- ✓ Similar al manejo tradicional de Futures

Contras:

- ✗ Menos composable
- ✗ Difícil de encadenar múltiples operaciones asíncronas

Opción B: `TaskEither` (Más Funcional)

`TaskEither` es un tipo que representa una operación asíncrona que puede fallar. Es más poderoso y composable.

```
import 'package:fpdart/fpdart.dart';

class GetUsers extends UseCase<List<User>, NoParams> {
  final UserRepository repository;

  GetUsers(this.repository);

  @override
  TaskEither<Failure, List<User>> call(NoParams params) {
    return TaskEither(() => repository.getUsers());
  }
}
```

Uso en el Cubit:


```
final result = await _getUsers(NoParams()).run();
result.match(
  (failure) => emit(UserError(failure.toString())),
  (users) => emit(UsersLoaded(users)),
);
```

Pros:

-  Más composable
-  Fácil de encadenar con `.andThen()`
-  Mejor para operaciones complejas
-  Más idiomático en programación funcional

Contras:

-  Curva de aprendizaje más alta
-  Puede ser overkill para casos simples

Ejemplo de Encadenamiento con TaskEither

```
// Encadenar múltiples operaciones
final program = _validateUser(id)
  .andThen((user) => _checkPermissions(user))
  .andThen((user) => _fetchUserDetails(user));

final result = await program.run();
```

Recomendación

Situación	Recomendación
Proyecto simple o aprendizaje	<code>Future<Either></code>
Proyecto complejo con múltiples operaciones asíncronas	<code>TaskEither</code>
Equipo con experiencia en FP	<code>TaskEither</code>
Necesitas compatibilidad con código existente	<code>Future<Either></code>

En esta guía usamos `Future<Either>` por ser más accesible, pero puedes migrar a `TaskEither` cuando te sientas cómodo.

5.2 Data Layer (La Cocina)

Model (con Isar)

Isar es una base de datos NoSQL de código abierto, extremadamente rápida y fácil de usar. A diferencia de Hive, Isar ofrece:

- Queries reactivos con streams
- Índices compuestos y multi-entrada
- Búsqueda de texto completo
- Completamente asíncrono con soporte multi-isolate

Es la opción recomendada para aplicaciones que necesitan alto rendimiento con datos locales complejos.

Archivo: `lib/features/user/data/models/user_model.dart`

```

import 'package:isar_community/isar_community.dart';
import 'package:my_app/features/user/domain/entities/user.dart';

part 'user_model.g.dart';

@collection
class UserModel {
  UserModel({
    this.id = Isar.autoIncrement,
    required this.name,
    required this.email,
    this.isActive = true,
    this.createdAt,
    this.avatarUrl,
  });

  Id id;

  @Index()
  String name;

  @Index(unique: true)
  String email;

  bool isActive;

  DateTime? createdAt;

  String? avatarUrl;

  User toEntity() {
    return User(
      id: id.toString(),
      name: name,
      email: email,
      isActive: isActive,
      createdAt: createdAt,
      avatarUrl: avatarUrl,
    );
  }

  factory UserModel.fromEntity(User entity) {
    return UserModel(
      id: entity.id.isEmpty ? int.tryParse(entity.id) ?? Isar.autoIncrement : Isar.autoIncrement,
      name: entity.name,
      email: entity.email,
      isActive: entity.isActive,
      createdAt: entity.createdAt,
      avatarUrl: entity.avatarUrl,
    );
  }
}

```

Diferencias clave Hive vs Isar:

Aspecto	Hive	Isar 3.x
Anotaciones	<code>@HiveType(typeId: 5)</code>	<code>@collection</code>
Campo ID	<code>String id</code>	<code>Id id = Isar.autoIncrement</code>
Índices	No tiene	<code>@Index()</code> , <code>@Index(unique: true)</code>
Herencia	Extiende <code>HiveObject</code>	No extiende nada
Serialización	Genera adapter	Genera schema

Model Nativo (API REST)

Cuando usas comunicación con APIs REST (JSON), no necesitas Isar para el modelo remote. Solo necesitas serialización con `fromJson` / `toJson`.

Archivo: `lib/features/user/data/models/user_model.dart`

```
import 'package:my_app/features/user/domain/entities/user.dart';

class UserModel extends User {
  const UserModel({
    required super.id,
    required super.name,
    required super.email,
    super.isActive,
    super.createdAt,
    super.avatarUrl,
  });

  factory UserModel.fromJson(Map<String, dynamic> json) {
    return UserModel(
      id: json['id']?.toString() ?? '',
      name: json['name'] ?? '',
      email: json['email'] ?? '',
      isActive: json['is_active'] ?? true,
      createdAt: json['created_at'] != null
        ? DateTime.parse(json['created_at'])
        : null,
      avatarUrl: json['avatar_url'],
    );
  }

  Map<String, dynamic> toJson() {
    return {
      'id': id,
      'name': name,
      'email': email,
      'is_active': isActive,
      'created_at': createdAt?.toIso8601String(),
      'avatar_url': avatarUrl,
    };
  }

  factory UserModel.fromEntity(User entity) {
    return UserModel(
      id: entity.id,
      name: entity.name,
      email: entity.email,
      isActive: entity.isActive,
      createdAt: entity.createdAt,
      avatarUrl: entity.avatarUrl,
    );
  }

  User toEntity() {
    return User(
      id: id,
      name: name,
      email: email,
      isActive: isActive,
      createdAt: createdAt,
      avatarUrl: avatarUrl,
    );
  }
}
```

Diferencias clave entre Isar y API REST:

Aspecto	Isar	API REST
Serialización	@collection + generación de schema	fromJson() / toJson()
Persistencia	Local en dispositivo	Remoto en servidor
Sincronización	No requiere red	Requiere conexión a internet
Offline	Soportado nativamente	Necesita caché local
Queries	Rich query language con índices	Limitado a endpoints del servidor
Reactivo	Streams para cambios	Polling o websockets

Remote DataSource — Dos opciones

El RemoteDataSource se implementa según el backend. Usa **Dio** para APIs REST externas, **SupabaseClient** cuando el backend es Supabase. El patrón es el mismo: el repositorio recibe un `remoteDataSource` y no sabe ni le importa cuál usa.

Opción 1: Dio (APIs REST externas)

Archivo: `lib/features/user/data/datasources/user_remote_data_source.dart`

```

import 'package:dio/dio.dart';
import 'package:my_app/core/error/exceptions.dart';
import 'package:my_app/features/user/data/models/user_model.dart';

abstract class UserRemoteDataSource {
  Future<List<UserModel>> getUsers();
  Future<UserModel> getUser(String id);
  Future<UserModel> createUser(UserModel user);
  Future<void> deleteUser(String id);
}

class UserRemoteDataSourceImpl implements UserRemoteDataSource {
  final Dio client;
  final String baseUrl;

  UserRemoteDataSourceImpl({
    required this.client,
    this.baseUrl = 'https://api.example.com',
  });

  @override
  Future<List<UserModel>> getUsers() async {
    try {
      final response = await client.get('$baseUrl/users');
      if (response.statusCode == 200) {
        final List<dynamic> jsonList = response.data;
        return jsonList.map((json) => UserModel.fromJson(json)).toList();
      } else {
        throw ServerException('Failed to load users: ${response.statusCode}');
      }
    } on DioException catch (e) {
      throw ServerException('Network error: ${e.message}');
    }
  }

  @override
  Future<UserModel> getUser(String id) async {
    try {
      final response = await client.get('$baseUrl/users/$id');
      if (response.statusCode == 200) {
        return UserModel.fromJson(response.data);
      } else {
        throw ServerException('User not found');
      }
    } on DioException catch (e) {
      throw ServerException('Network error: ${e.message}');
    }
  }

  @override
  Future<UserModel> createUser(UserModel user) async {
    try {
      final response = await client.post(
        '$baseUrl/users',
        data: user.toJson(),
      );
      if (response.statusCode == 201) {
        return UserModel.fromJson(response.data);
      } else {
        throw ServerException('Failed to create user');
      }
    } on DioException catch (e) {
      throw ServerException('Network error: ${e.message}');
    }
  }

  @override
  Future<void> deleteUser(String id) async {

```

```
    try {  
      await client.delete('$baseUrl/users/$id');  
    } on DioException catch (e) {  
      throw ServerException('Network error: ${e.message}');  
    }  
  }  
}
```

Opción 2: SupabaseClient (backend Supabase)

Archivo: `lib/features/user/data/datasources/user_remote_data_source.dart`

```

import 'package:supabase_flutter/supabase_flutter.dart';
import 'package:my_app/core/error/exceptions.dart';
import 'package:my_app/features/user/data/models/user_model.dart';

abstract class UserRemoteDataSource {
  Future<List<UserModel>> getUsers();
  Future<UserModel> getUser(String id);
  Future<UserModel> createUser(UserModel user);
  Future<void> deleteUser(String id);
}

class UserRemoteDataSourceImpl implements UserRemoteDataSource {
  final SupabaseClient supabase;

  UserRemoteDataSourceImpl({required this.supabase});

  @override
  Future<List<UserModel>> getUsers() async {
    try {
      final response = await supabase
        .from('users')
        .select()
        .order('created_at', ascending: false);
      return (response as List)
        .map((json) => UserModel.fromJson(json))
        .toList();
    } on PostgrestException catch (e) {
      throw ServerException(e.message);
    }
  }

  @override
  Future<UserModel> getUser(String id) async {
    try {
      final response = await supabase
        .from('users')
        .select()
        .eq('id', id)
        .maybeSingle();
      if (response == null) throw ServerException('User not found');
      return UserModel.fromJson(response);
    } on PostgrestException catch (e) {
      throw ServerException(e.message);
    }
  }

  @override
  Future<UserModel> createUser(UserModel user) async {
    try {
      final response = await supabase
        .from('users')
        .insert(user.toJson())
        .select()
        .single();
      return UserModel.fromJson(response);
    } on PostgrestException catch (e) {
      throw ServerException(e.message);
    }
  }

  @override
  Future<void> deleteUser(String id) async {
    try {
      await supabase.from('users').delete().eq('id', id);
    } on PostgrestException catch (e) {
      throw ServerException(e.message);
    }
  }
}

```



```
}  
}
```

¿Cuál usar?

Criterio	Dio	SupabaseClient
Backend propio (REST API)	✓	✗
Backend Supabase	✗	✓
Microservicios externos	✓	✗
Auth + DB + Storage + Realtime	✗	✓
Control fino de headers/cache	✓	⚠ Limitado
Testeo con mocks	Dio Mock	supabase mocks

NetworkInfo

Para manejar la lógica online/offline, necesitas verificar si hay conexión a internet.

Archivo: `lib/core/network/network_info.dart`

```
import 'package:internet_connection_checker_plus/internet_connection_checker_plus.dart';  
  
abstract class NetworkInfo {  
  Future<bool> get isConnected;  
}  
  
class NetworkInfoImpl implements NetworkInfo {  
  final InternetConnection connectionChecker;  
  
  NetworkInfoImpl(this.connectionChecker);  
  
  @override  
  Future<bool> get isConnected => connectionChecker.hasInternetAccess;  
  
  @override  
  Stream<bool> get onConnectivityChanged =>  
    connectionChecker.onStatusChange.map(  
      (status) => status == InternetStatus.connected,  
    );  
}
```

Exceptions

Archivo: `lib/core/error/exceptions.dart`

```
class ServerException implements Exception {
  final String message;
  ServerException(this.message);

  @override
  String toString() => message;
}

class CacheException implements Exception {
  final String message;
  CacheException(this.message);

  @override
  String toString() => message;
}

class AuthException implements Exception {
  final String message;
  final int? statusCode;

  AuthException({required this.message, this.statusCode});

  @override
  String toString() => 'AuthException: $message (status: $statusCode)';
}
```

DataSource (con Isar)

Archivo: `lib/features/user/data/datasources/user_local_data_source.dart`

```
import 'package:isar_community/isar_community.dart';
import 'package:my_app/features/user/data/models/user_model.dart';

abstract class UserLocalDataSource {
  Future<List<UserModel>> getUsers();
  Future<UserModel?> getUser(int id);
  Future<int> saveUser(UserModel user);
  Future<void> deleteUser(int id);
}

class UserLocalDataSourceImpl implements UserLocalDataSource {
  final Isar _isar;

  UserLocalDataSourceImpl(this._isar);

  @override
  Future<List<UserModel>> getUsers() async {
    return _isar.userModels.where().findAll();
  }

  @override
  Future<UserModel?> getUser(int id) async {
    return _isar.userModels.get(id);
  }

  @override
  Future<int> saveUser(UserModel user) async {
    return _isar.writeTxn(() async {
      return _isar.userModels.put(user);
    });
  }

  @override
  Future<void> deleteUser(int id) async {
    await _isar.writeTxn(() async {
      await _isar.userModels.delete(id);
    });
  }
}
```

Diferencias clave entre Hive Box e Isar:

Aspecto	Hive Box	Isar
Obtener todos	<code>_box.values.toList()</code>	<code>_isar.userModels.where().findAll()</code>
Obtener uno	<code>_box.get(id)</code>	<code>_isar.userModels.get(id)</code>
Guardar	<code>_box.put(key, value)</code>	<code>_isar.writeTxn(() => isar.collection.put(value))</code>
Eliminar	<code>_box.delete(key)</code>	<code>_isar.writeTxn(() => isar.collection.delete(id))</code>
Tipo ID	String	int (Id)

Repository Implementation (con lógica Online/Offline)

Este repository decide automáticamente si usar datos remotos (API) o locales (caché) según la conexión.
 Recibe **4 dependencias estándar**: `remoteDataSource`, `localDataSource`, `networkInfo` y `userSession`.

Archivo: `lib/features/user/data/repositories/user_repository_impl.dart`

```

import 'package:fpdart/fpdart.dart';
import 'package:my_app/core/error/exceptions.dart';
import 'package:my_app/core/error/failures.dart';
import 'package:my_app/core/network/network_info.dart';
import 'package:my_app/core/session/user_session.dart';
import 'package:my_app/features/user/data/datasources/user_local_data_source.dart';
import 'package:my_app/features/user/data/datasources/user_remote_data_source.dart';
import 'package:my_app/features/user/data/models/user_model.dart';
import 'package:my_app/features/user/domain/entities/user.dart';
import 'package:my_app/features/user/domain/repositories/user_repository.dart';

class UserRepositoryImpl implements UserRepository {
  final UserRemoteDataSource remoteDataSource;
  final UserLocalDataSource localDataSource;
  final NetworkInfo networkInfo;
  final UserSession userSession;

  UserRepositoryImpl({
    required this.remoteDataSource,
    required this.localDataSource,
    required this.networkInfo,
    required this.userSession,
  });

  @override
  Future<Either<Failure, List<User>>> getUsers() async {
    if (await networkInfo.isConnected) {
      try {
        final remoteUsers = await remoteDataSource.getUsers();
        await _cacheUsers(remoteUsers);
        return Right(remoteUsers.map((m) => m.toEntity()).toList());
      } on ServerException catch (e) {
        return Left(ServerFailure(e.message));
      }
    } else {
      return _getUsersFromCache();
    }
  }

  Future<Either<Failure, List<User>>> _getUsersFromCache() async {
    try {
      final localUsers = await localDataSource.getUsers();
      return Right(localUsers.map((m) => m.toEntity()).toList());
    } catch (e) {
      return Left(CacheFailure('No cached data available'));
    }
  }

  Future<void> _cacheUsers(List<UserModel> users) async {
    for (final user in users) {
      await localDataSource.saveUser(user);
    }
  }

  @override
  Future<Either<Failure, User>> getUser(String id) async {
    if (await networkInfo.isConnected) {
      try {
        final remoteUser = await remoteDataSource.getUser(id);
        await localDataSource.saveUser(remoteUser);
        return Right(remoteUser);
      } on ServerException catch (e) {
        return Left(ServerFailure(e.message));
      }
    } else {
      try {
        final localUser = await localDataSource.getUser(id);
        if (localUser == null) {

```

```

        return Left(CacheFailure('User not found in cache'));
    }
    return Right(localUser);
} catch (e) {
    return Left(CacheFailure(e.toString()));
}
}

@override
Future<Either<Failure, void>> createUser(User user) async {
    if (await networkInfo.isConnected) {
        try {
            final userModel = UserModel.fromEntity(user);
            await remoteDataSource.createUser(userModel);
            await localDataSource.saveUser(userModel);
            return Right(null);
        } on ServerException catch (e) {
            return Left(ServerFailure(e.message));
        }
    } else {
        return Left(NetworkFailure('Cannot create user offline'));
    }
}

@override
Future<Either<Failure, void>> deleteUser(String id) async {
    if (await networkInfo.isConnected) {
        try {
            await remoteDataSource.deleteUser(id);
            await localDataSource.deleteUser(id);
            return Right(null);
        } on ServerException catch (e) {
            return Left(ServerFailure(e.message));
        }
    } else {
        return Left(NetworkFailure('Cannot delete user offline'));
    }
}
}

```

Uso de `userSession`: El `userId` se usa para filtrar datos del usuario autenticado. Ejemplo: `final userId = userSession.userId;` (síncrono).

Variante: Repositorio con múltiples Remote DataSources

A veces un repositorio necesita combinar **dos o más** fuentes remotas. Por ejemplo, un repositorio de sorteos que necesita datos de `RaffleRemoteDataSource` y `TicketRemoteDataSource`:

```

class RaffleRepositoryImpl implements RaffleRepository {
    final RaffleRemoteDataSource remoteDataSource;
    final TicketRemoteDataSource ticketRemoteDataSource;
    final NetworkInfo networkInfo;
    final UserSession userSession;

    RaffleRepositoryImpl({
        required this.remoteDataSource,
        required this.ticketRemoteDataSource,
        required this.networkInfo,
        required this.userSession,
    });

    @override
    Future<Either<Failure, Raffle>> createRaffle(Raffle raffle) async {
        final userId = userSession.userId;
        if (userId == null) return Left(AuthFailure());

        if (await networkInfo.isConnected) {
            try {
                final model = await remoteDataSource.createRaffle(
                    RaffleModel.fromEntity(raffle),
                );
                return Right(model.toEntity());
            } on ServerException catch (e) {
                return Left(ServerFailure(e.message));
            }
        } else {
            return Left(NetworkFailure('Cannot create raffle offline'));
        }
    }

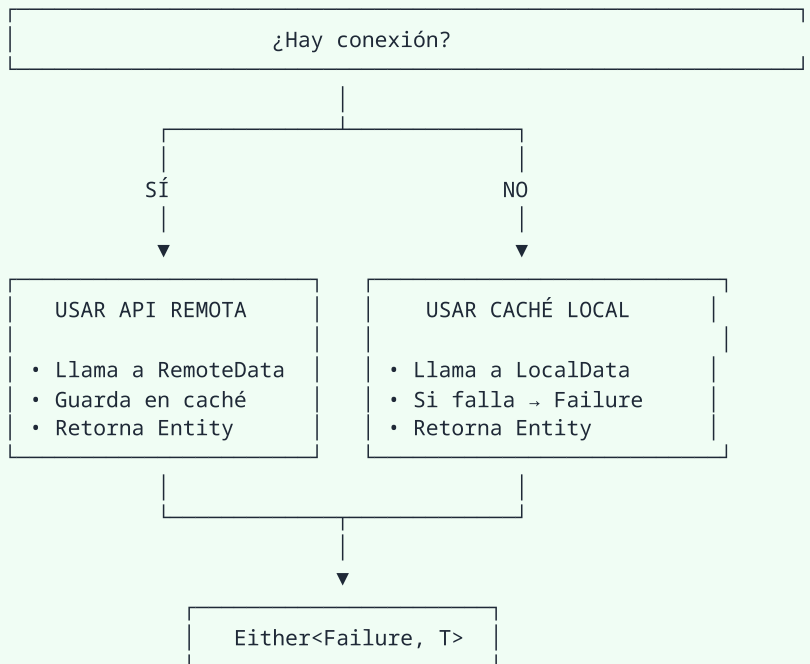
    @override
    Future<Either<Failure, List<Ticket>>> getTickets(String raffleId) async {
        if (await networkInfo.isConnected) {
            try {
                final tickets = await ticketRemoteDataSource.getTickets(raffleId);
                return Right(tickets.map((m) => m.toEntity()).toList());
            } on ServerException catch (e) {
                return Left(ServerFailure(e.message));
            }
        } else {
            return Left(NetworkFailure('Cannot load tickets offline'));
        }
    }
}

```

¿Cuándo incluir `UserSession`?

Repositorio	¿Necesita UserSession?	Motivo
User/Profile	✓ Sí	Filtra datos por <code>userId</code>
PaymentMethod	✓ Sí	Asocia métodos de pago al usuario
Raffle (sorteo)	✓ Sí	Crea recursos pertenecientes al usuario
Ticket	✓ Sí	Reserva/compra asociada al usuario
Auth	✗ No	Usa <code>localDataSource.hasCachedUser</code> directamente
Winner (público)	✗ No	Consulta pública sin dueño

Lógica de decisión del Repository:



Flujo completo de getUsers():

```
getUsers()
├── ¿isConnected?
│   ├── SÍ → remoteDataSource.getUsers()
│   │   ├── ÉXITO → cacheUsers() → return Either.right(users)
│   │   └── ERROR → return Either.left(ServerFailure)
│   └── NO → localDataSource.getUsers()
│       ├── ÉXITO → return Either.right(users)
│       └── ERROR → return Either.left(CacheFailure)
```

Nota: Con Isar, puedes aprovechar sus streams reactivos para observadores en tiempo real de los datos locales, lo cual simplifica la sincronización con la UI.

5.3 Presentation Layer (El Mesero)

States

Archivo: `lib/features/user/presentation/cubit/user_state.dart`

```
part of 'user_cubit.dart';

abstract class UserState extends Equatable {
  const UserState();

  @override
  List<Object?> get props => [];
}

class UserInitial extends UserState {}
class UserLoading extends UserState {}

class UsersLoaded extends UserState {
  final List<User> users;
  const UsersLoaded(this.users);

  @override
  List<Object?> get props => [users];
}

class UserLoaded extends UserState {
  final User user;
  const UserLoaded(this.user);

  @override
  List<Object?> get props => [user];
}

class UserError extends UserState {
  final String message;
  const UserError(this.message);

  @override
  List<Object?> get props => [message];
}

class UserOperationSuccess extends UserState {
  final String message;
  const UserOperationSuccess(this.message);

  @override
  List<Object?> get props => [message];
}
```

Cubit

Archivo: `lib/features/user/presentation/cubit/user_cubit.dart`


```

import 'package:bloc/bloc.dart';
import 'package:equatable/equatable.dart';
import 'package:my_app/core/common/usecase.dart';
import 'package:my_app/features/user/domain/entities/user.dart';
import 'package:my_app/features/user/domain/usecases/create_user.dart';
import 'package:my_app/features/user/domain/usecases/delete_user.dart';
import 'package:my_app/features/user/domain/usecases/get_user.dart';
import 'package:my_app/features/user/domain/usecases/get_users.dart';

part 'user_state.dart';

class UserCubit extends Cubit<UserState> {
  final GetUsers _getUsers;
  final GetUser _getUser;
  final CreateUser _createUser;
  final DeleteUser _deleteUser;

  UserCubit({
    required GetUsers getUsers,
    required GetUser getUser,
    required CreateUser createUser,
    required DeleteUser deleteUser,
  }) : _getUsers = getUsers,
       _getUser = getUser,
       _createUser = createUser,
       _deleteUser = deleteUser,
       super(UserInitial());

  Future<void> loadUsers() async {
    emit(UserLoading());

    final result = await _getUsers(NoParams());

    result.match(
      (failure) => emit(UserError(failure.toString())),
      (users) => emit(UsersLoaded(users)),
    );
  }

  Future<void> loadUser(String id) async {
    emit(UserLoading());

    final result = await _getUser(GetUserParams(id));

    result.match(
      (failure) => emit(UserError(failure.toString())),
      (user) => emit(UserLoaded(user)),
    );
  }

  Future<void> createUser(String name, String email) async {
    emit(UserLoading());

    final result = await _createUser(
      CreateUserParams(name: name, email: email),
    );

    result.match(
      (failure) => emit(UserError(failure.toString())),
      (_) {
        emit(const UserOperationSuccess('User created'));
        loadUsers();
      },
    );
  }

  Future<void> deleteUser(String userId) async {
    emit(UserLoading());
  }

```

```
final result = await _deleteUser(DeleteUserParams(userId));

result.match(
  (failure) => emit(UserError(failure.toString())),
  (_) {
    emit(const UserOperationSuccess('User deleted'));
    loadUsers();
  },
);
}
```

📖 **Para una guía completa de BLoC/Cubit** (widgets, concurrencia, hydrated, testing, buenas prácticas, proyecto integrador): módulo [16-BLOC-CUBIT/](#).

5.4 UI Layer (El Cliente)

Users List Page

Archivo: [lib/features/user/presentation/pages/users_list_page.dart](#)

```

import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:get_it/get_it.dart';
import 'package:my_app/features/user/presentation/cubit/user_cubit.dart';

class UsersListPage extends StatelessWidget {
  const UsersListPage({super.key});

  @override
  Widget build(BuildContext context) {
    return BlocProvider(
      create: (_) => GetIt.I<UserCubit>()..loadUsers(),
      child: Scaffold(
        appBar: AppBar(title: const Text('Users')),
        body: const _UsersListView(),
        floatingActionButton: FloatingActionButton(
          onPressed: () => _showCreateDialog(context),
          child: const Icon(Icons.add),
        ),
      ),
    );
  }

  void _showCreateDialog(BuildContext context) {
    showDialog(
      context: context,
      builder: (dialogContext) => BlocProvider.value(
        value: context.read<UserCubit>(),
        child: const _CreateUserDialog(),
      ),
    );
  }
}

class _UsersListView extends StatelessWidget {
  const _UsersListView();

  @override
  Widget build(BuildContext context) {
    return BlocConsumer<UserCubit, UserState>(
      listener: (context, state) {
        if (state is UserOperationSuccess) {
          ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(content: Text(state.message)),
          );
        }
      },
      builder: (context, state) {
        if (state is UserLoading) {
          return const Center(child: CircularProgressIndicator());
        }

        if (state is UsersLoaded) {
          if (state.users.isEmpty) {
            return const Center(child: Text('No users yet'));
          }

          return ListView.builder(
            itemCount: state.users.length,
            itemBuilder: (context, index) {
              final user = state.users[index];
              return ListTile(
                leading: CircleAvatar(
                  child: Text(user.name[0].toUpperCase()),
                ),
                title: Text(user.name),
                subtitle: Text(user.email),
                trailing: IconButton(

```

```

        icon: const Icon(Icons.delete, color: Colors.red),
        onPressed: () => _confirmDelete(context, user.id),
      ),
      onTap: () => Navigator.push(
        context,
        MaterialPageRoute(
          builder: (_) => BlocProvider.value(
            value: context.read<UserCubit>(),
            child: UserDetailPage(userId: user.id),
          ),
        ),
      ),
    );
  },
);
}

if (state is UserError) {
  return Center(child: Text(state.message));
}

return const SizedBox.shrink();
},
);
}

void _confirmDelete(BuildContext context, String userId) {
  showDialog(
    context: context,
    builder: (dialogContext) => AlertDialog(
      title: const Text('Delete User'),
      content: const Text('Are you sure you want to delete this user?'),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(dialogContext),
          child: const Text('Cancel'),
        ),
        TextButton(
          onPressed: () {
            Navigator.pop(dialogContext);
            context.read<UserCubit>().deleteUser(userId);
          },
          child: const Text('Delete', style: TextStyle(color: Colors.red)),
        ),
      ],
    ),
  );
}

class _CreateUserDialog extends StatefulWidget {
  const _CreateUserDialog();

  @override
  State<_CreateUserDialog> createState() => _CreateUserDialogState();
}

class _CreateUserDialogState extends State<_CreateUserDialog> {
  final _nameController = TextEditingController();
  final _emailController = TextEditingController();

  @override
  void dispose() {
    _nameController.dispose();
    _emailController.dispose();
    super.dispose();
  }

  @override

```

```

Widget build(BuildContext context) {
  return AlertDialog(
    title: const Text('New User'),
    content: Column(
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        TextField(
          controller: _nameController,
          decoration: const InputDecoration(labelText: 'Name'),
        ),
        TextField(
          controller: _emailController,
          decoration: const InputDecoration(labelText: 'Email'),
          keyboardType: TextInputType.emailAddress,
        ),
      ],
    ),
    actions: [
      TextButton(
        onPressed: () => Navigator.pop(context),
        child: const Text('Cancel'),
      ),
      TextButton(
        onPressed: () {
          final name = _nameController.text;
          final email = _emailController.text;
          if (name.isNotEmpty && email.isNotEmpty) {
            context.read<UserCubit>().createUser(name, email);
            Navigator.pop(context);
          }
        },
        child: const Text('Create'),
      ),
    ],
  );
}

```

User Detail Page

Archivo: `lib/features/user/presentation/pages/user_detail_page.dart`

```

import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:get_it/get_it.dart';
import 'package:my_app/features/user/presentation/cubit/user_cubit.dart';

class UserDetailPage extends StatelessWidget {
  final String userId;

  const UserDetailPage({super.key, required this.userId});

  @override
  Widget build(BuildContext context) {
    return BlocProvider(
      create: (_) => GetIt.I<UserCubit>().loadUser(userId),
      child: Scaffold(
        appBar: AppBar(title: const Text('User Details')),
        body: const _UserDetailView(),
      ),
    );
  }
}

class _UserDetailView extends StatelessWidget {
  const _UserDetailView();

  @override
  Widget build(BuildContext context) {
    return BlocBuilder<UserCubit, UserState>(
      builder: (context, state) {
        if (state is UserLoading) {
          return const Center(child: CircularProgressIndicator());
        }

        if (state is UserLoaded) {
          final user = state.user;
          return Padding(
            padding: const EdgeInsets.all(16.0),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Center(
                  child: CircleAvatar(
                    radius: 50,
                    child: Text(
                      user.name[0].toUpperCase(),
                      style: const TextStyle(fontSize: 40),
                    ),
                  ),
                ),
                const SizedBox(height: 24),
                _DetailRow(label: 'Name', value: user.name),
                _DetailRow(label: 'Email', value: user.email),
                _DetailRow(
                  label: 'Active',
                  value: user.isActive ? 'Yes' : 'No',
                ),
                if (user.createdAt != null)
                  _DetailRow(
                    label: 'Created',
                    value: user.createdAt!.toString().split(' ')[0],
                  ),
                if (user.isNew)
                  const Chip(
                    label: Text('NEW'),
                    backgroundColor: Colors.green,
                  ),
              ],
            ),
          );
        }
      },
    );
  }
}

```

```

    );
  }

  if (state is UserError) {
    return Center(child: Text(state.message));
  }

  return const SizedBox.shrink();
},
);
}
}

class _DetailRow extends StatelessWidget {
  final String label;
  final String value;

  const _DetailRow({required this.label, required this.value});

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 8.0),
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          SizedBox(
            width: 100,
            child: Text(
              '$label:',
              style: const TextStyle(fontWeight: FontWeight.bold),
            ),
          ),
          Expanded(child: Text(value)),
        ],
      ),
    );
  }
}

```

6. Inyección de Dependencias con GetIt

¿Qué es la Inyección de Dependencias?

Imagina que estás construyendo una casa. Necesitas un electricista, un plomero y un carpintero. En lugar de que cada uno fabrication sus propias herramientas, **tú les proporcionas las herramientas que necesitan**.

En programación:

- Una **dependencia** es cualquier objeto que tu clase necesita para funcionar
- **Injectar** significa proporcionar esos objetos desde afuera

¿Por qué es importante?

1. **Desacoplamiento**: Las clases no dependen de implementaciones específicas
2. **Testabilidad**: Puedes inyectar "mocks" para probar tu código
3. **Flexibilidad**: Puedes cambiar implementaciones sin modificar el código

El Problema: Constructor Drilling

Sin inyección de dependencias centralizada:

```
// × SIN inyección centralizada - MUY TEDIOSO
void main() {
  final localDataSource = UserLocalDataSourceImpl(box: box);
  final repository = UserRepositoryImpl(localDataSource: localDataSource);
  final getUsers = GetUsers(repository);
  final createUser = CreateUser(repository);
  final cubit = UserCubit(
    getUsers: getUsers,
    getUser: getUser,
    createUser: createUser,
    deleteUser: deleteUser,
  );
}
```

Solución: GetIt (Service Locator)

GetIt es un paquete que implementa el patrón **Service Locator**. Hay dos estilos: **manual** (cascade notation) y **automático** (con `injectable`). Ambos son válidos; el manual es más explícito y el automático reduce boilerplate.

Conceptos clave de GetIt

Método	Descripción	Cuándo usarlo
<code>registerSingleton()</code>	Crea la instancia inmediatamente	Para objetos que deben existir desde el inicio
<code>registerLazySingleton()</code>	Crea la instancia la primera vez que se use	Para objetos pesados que quizás no se usen (recomendado)
<code>registerFactory()</code>	Crea una nueva instancia CADA vez que se pida	Para objetos que no deben compartir estado (como Cubits)

Estilo 1: Manual con cascade notation (recomendado)

Cada dependencia se registra explícitamente con `sl.registerLazySingleton<T>(() => T(...))`, usando `cascade (..)` para agrupar por capa.

Archivo: `lib/core/di/service_locator.dart`

```

import 'package:dio/dio.dart';
import 'package:get_it/get_it.dart';
import 'package:internet_connection_checker_plus/internet_connection_checker_plus.dart';
import 'package:isar_community/isar.dart';
import 'package:path_provider/path_provider.dart';
import 'package:supabase_flutter/supabase_flutter.dart';
import 'package:my_app/core/data/local/isar_service.dart';
import 'package:my_app/core/network/network_info.dart';
import 'package:my_app/core/services/cache_manager.dart';
import 'package:my_app/core/session/user_session.dart';
import 'package:my_app/core/session/user_session_impl.dart';
import 'package:my_app/features/user/data/datasources/user_local_data_source.dart';
import 'package:my_app/features/user/data/datasources/user_remote_data_source.dart';
import 'package:my_app/features/user/data/models/user_model.dart';
import 'package:my_app/features/user/data/repositories/user_repository_impl.dart';
import 'package:my_app/features/user/domain/repositories/user_repository.dart';
import 'package:my_app/features/user/domain/usecases/create_user.dart';
import 'package:my_app/features/user/domain/usecases/delete_user.dart';
import 'package:my_app/features/user/domain/usecases/get_user.dart';
import 'package:my_app/features/user/domain/usecases/get_users.dart';
import 'package:my_app/features/user/presentation/cubit/user_cubit.dart';

```

```

final sl = GetIt.instance;

```

```

Future<void> initDependencies() async {
  await IsarService.initialize();

```

```

  sl

```

```

  //
  // CAPA EXTERNA - Librerías de terceros
  //

```

```

  ..registerLazySingleton<Isar>(() => IsarService.instance)
  ..registerLazySingleton<InternetConnection>(
    InternetConnection.createInstance,
  )
  ..registerLazySingleton<Dio>(() {
    final dio = Dio(BaseOptions(
      connectTimeout: const Duration(seconds: 30),
      receiveTimeout: const Duration(seconds: 30),
      headers: {'Content-Type': 'application/json'},
    ));
    dio.interceptors.add(LogInterceptor(
      requestBody: true,
      responseBody: true,
    ));
    return dio;
  })

```

```

  //
  // CORE - Network, Session, Cache
  //

```

```

  ..registerLazySingleton<NetworkInfo>(
    () => NetworkInfoImpl(sl<InternetConnection>()),
  )
  ..registerLazySingleton<UserSession>(
    () => UserSessionImpl(sl<Isar>(), supabase: Supabase.instance.client),
  )
  ..registerLazySingleton<CacheManager>(CacheManager.new)

```

```

  //
  // CAPA DE DATOS - DataSources
  //

```

```

  ..registerLazySingleton<UserRemoteDataSource>(
    () => UserRemoteDataSourceImpl(client: sl<Dio>()),
  )
  ..registerLazySingleton<UserLocalDataSource>(
    () => UserLocalDataSourceImpl(sl<Isar>()),
  )

```

```

//
// CAPA DE REPOSITORIO
//
..registerLazySingleton<UserRepository>(
    () => UserRepositoryImpl(
        remoteDataSource: sl<UserRemoteDataSource>(),
        localDataSource: sl<UserLocalDataSource>(),
        networkInfo: sl<NetworkInfo>(),
        userSession: sl<UserSession>(),
    ),
)

//
// CAPA DE DOMINIO - UseCases
//
..registerLazySingleton(() => GetUsers(sl()))
..registerLazySingleton(() => GetUser(sl()))
..registerLazySingleton(() => CreateUser(sl()))
..registerLazySingleton(() => DeleteUser(sl()))

//
// CAPA DE PRESENTACIÓN - Cubit
//
..registerFactory(() => UserCubit(
    getUsers: sl(),
    getUser: sl(),
    createUser: sl(),
    deleteUser: sl(),
));

// Registrar limpieza de cachés en CacheManager
sl<CacheManager>()
    ..register(() => sl<UserLocalDataSource>().clearCache());
}

```

Cascade notation (`..`): permite llamar múltiples métodos sobre el mismo objeto sin repetir `sl.` cada vez. Cada `..registerLazySingleton` opera sobre el mismo `sl`.

Registro de repositorios con UserSession

Cuando el repositorio requiere `UserSession`, se pasa explícitamente:

```

..registerLazySingleton<ProfileRepository>(
    () => ProfileRepositoryImpl(
        remoteDataSource: sl<ProfileRemoteDataSource>(),
        localDataSource: sl<ProfileLocalDataSource>(),
        networkInfo: sl<NetworkInfo>(),
        userSession: sl<UserSession>(),
    ),
)

```

Estilo 2: Automático con injectable (para proyectos grandes)

Usa código generado para reducir boilerplate. Ideal cuando tienes 20+ features. Tus clases se anotan y `injectable` genera el registro automático. Requiere `build_runner`.

```
# pubspec.yaml
dependencies:
  get_it: ^8.0.3
  injectable: ^2.5.0

dev_dependencies:
  injectable_generator: ^2.7.0
  build_runner: ^2.4.15
```

Archivo: `lib/core/di/injection_container.dart`

```
import 'package:get_it/get_it.dart';
import 'package:injectable/injectable.dart';
import 'package:my_app/core/di/injection_container.config.dart';

final getIt = GetIt.instance;

@InjectableInit(
  initializerName: r'$initGetIt',
  preferRelativeImports: true,
  asExtension: false,
)
Future<void> configureDependencies() async {
  await $initGetIt(getIt);
}
```

Luego cada clase se anota:

```
@lazySingleton
class UserRemoteDataSourceImpl implements UserRemoteDataSource {
  final Dio client;
  UserRemoteDataSourceImpl(this.client);
  // ...
}

@lazySingleton
class UserRepositoryImpl implements UserRepository {
  final UserRemoteDataSource remoteDataSource;
  final UserLocalDataSource localDataSource;
  final NetworkInfo networkInfo;
  final UserSession userSession;

  UserRepositoryImpl(
    this.remoteDataSource,
    this.localDataSource,
    this.networkInfo,
    this.userSession,
  );
  // ...
}
```

¿Cuál elegir?

Criterio	Manual	Injectable
Boilerplate	Más código escrito	Menos código (generado)
Visibilidad	Explícito, fácil de debuggear	Oculto tras código generado
Dependencia extra	No	<code>injectable</code> + generator + build_runner
Ideal para	Proyectos pequeños/medios (< 15 features)	Proyectos grandes (20+ features)
Control fino	Total	Limitado por anotaciones

Criterio	Manual	Injectable
Tiempo de compilación	Sin impacto	build_runner en cada cambio

Nota: Si usas **solo** base de datos local sin API, el `remoteDataSource` no es necesario. En ese caso, el `UserRepositoryImpl` solo usaría `localDataSource`.

Versión con solo Isar (sin API REST):

```
// Versión simplificada solo con base de datos local Isar
Future<void> init() async {
  // Isar - Base de datos local
  final dir = await getApplicationDocumentsDirectory();
  final isar = await Isar.open(
    [UserModelSchema],
    directory: dir.path,
  );
  sl.registerLazySingleton<Isar>(() => isar);

  // LocalDataSource (Isar)
  sl.registerLazySingleton<UserLocalDataSource>(
    () => UserLocalDataSourceImpl(sl()),
  );

  // Repository (solo con localDataSource)
  sl.registerLazySingleton<UserRepository>(
    () => UserRepositoryImpl(
      localDataSource: sl(),
    ),
  );

  // ... resto de UseCases y Cubits
}
```

Diferencia clave: El `UserRepositoryImpl` necesitaría una versión simplificada que solo use `localDataSource` sin verificar conexión a internet.

Archivo: `lib/core/common/usecase.dart`

```
import 'package:fpdart/fpdart.dart';
import 'package:equatable/equatable.dart';
import 'package:my_app/core/error/failures.dart';

abstract class UseCase<Type, Params> {
  Future<Either<Failure, Type>> call(Params params);
}

class NoParams extends Equatable {
  @override
  List<Object?> get props => [];
}
```

Archivo: `lib/core/error/failures.dart`

```
import 'package:equatable/equatable.dart';

abstract class Failure extends Equatable {
  final String message;

  const Failure(this.message);

  @override
  List<Object?> get props => [message];
}

class CacheFailure extends Failure {
  const CacheFailure(super.message);
}

class ServerFailure extends Failure {
  const ServerFailure(super.message);
}

class NetworkFailure extends Failure {
  const NetworkFailure(super.message);
}
```

Archivo: `lib/main.dart`

```
import 'package:flutter/material.dart';
import 'package:my_app/core/di/injection_container.dart' as di;
import 'package:my_app/features/user/presentation/pages/users_list_page.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await di.init();
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Clean Architecture Demo',
      home: const UsersListPage(),
    );
  }
}
```

8. Templates Universales

Template 1: Entity

```
import 'package:equatable/equatable.dart';

class {Feature} extends Equatable {
  const {Feature}({
    required this.id,
    required this.name,
    this.isActive = true,
    this.createdAt,
  });

  final String id;
  final String name;
  final bool isActive;
  final DateTime? createdAt;

  {Feature} copyWith({
    String? id,
    String? name,
    bool? isActive,
    DateTime? createdAt,
  }) {
    return {Feature}({
      id: id ?? this.id,
      name: name ?? this.name,
      isActive: isActive ?? this.isActive,
      createdAt: createdAt ?? this.createdAt,
    });
  }

  @override
  List<Object?> get props => [id, name, isActive, createdAt];
}
```

Template 2: Repository Interface

```
import 'package:fpdart/fpdart.dart';
import 'package:my_app/core/error/failures.dart';
import 'package:my_app/features/{feature}/domain/entities/{feature}.dart';

abstract class {Feature}Repository {
  Future<Either<Failure, List<{Feature}>>>> getAll();
  Future<Either<Failure, {Feature}>>> getById(String id);
  Future<Either<Failure, void>>> create({Feature} {feature});
  Future<Either<Failure, void>>> update({Feature} {feature});
  Future<Either<Failure, void>>> delete(String id);
}
```

Template 3: UseCase

```
import 'package:fpdart/fpdart.dart';
import 'package:equatable/equatable.dart';
import 'package:my_app/core/common/usecase.dart';
import 'package:my_app/core/error/failures.dart';
import 'package:my_app/features/{feature}/domain/entities/{feature}.dart';
import 'package:my_app/features/{feature}/domain/repositories/{feature}_repository.dart';

class Get{Feature} extends UseCase<{Feature}, Get{Feature}Params> {
  final {Feature}Repository repository;

  Get{Feature}(this.repository);

  @override
  Future<Either<Failure, {Feature}>> call(Get{Feature}Params params) async {
    return await repository.getById(params.id);
  }
}

class Get{Feature}Params extends Equatable {
  final String id;

  const Get{Feature}Params(this.id);

  @override
  List<Object?> get props => [id];
}
```

Template 4: Cubit

```
import 'package:bloc/bloc.dart';
import 'package:equatable/equatable.dart';
import 'package:my_app/core/common/usecase.dart';
import 'package:my_app/features/{feature}/domain/entities/{feature}.dart';
import 'package:my_app/features/{feature}/domain/usecases/get_{feature}.dart';

part '{feature}_state.dart';

class {Feature}Cubit extends Cubit<{Feature}State> {
  final Get{Feature} _get{Feature};

  {Feature}Cubit({required Get{Feature} get{Feature}})
    : _get{Feature} = get{Feature},
      super({Feature}Initial());

  Future<void> load{Feature}(String id) async {
    emit({Feature}Loading());

    final result = await _get{Feature}(Get{Feature}Params(id));

    result.match(
      (failure) => emit({Feature}Error(failure.toString())),
      ({feature}) => emit({Feature}Loaded({feature})),
    );
  }
}
```


Template 5: State

```
part of '{feature}_cubit.dart';

abstract class {Feature}State extends Equatable {
  const {Feature}State();

  @override
  List<Object?> get props => [];
}

class {Feature}Initial extends {Feature}State {}
class {Feature>Loading extends {Feature}State {}
class {Feature}Loaded extends {Feature}State {
  final {Feature} {feature};
  const {Feature}Loaded(this.{feature});

  @override
  List<Object?> get props => [{feature}];
}
class {Feature}Error extends {Feature}State {
  final String message;
  const {Feature}Error(this.message);

  @override
  List<Object?> get props => [message];
}
```

 **Buenas prácticas de BLoC/Cubit** (naming, BlocObserver, anti-patrones, arquitectura): [16-BLOC-CUBIT/16-buenas-practicas.md](#).

9. Decisiones de Arquitectura

Cubit vs BLoC

Aspecto	Cubit	BLoC
Complejidad	Más simple	Más verboso
Uso	Funciones directas	Eventos específicos
Trazabilidad	Menor	Mayor
Ideal para	Mayoría de casos	Lógica compleja con múltiples eventos

Recomendación: Empieza con `Cubit`. Si la lógica se vuelve muy compleja, refactoriza a `BLoC`.

¿Dónde va cada cosa?

Situación	¿Dónde va?	Ejemplo
Validar formato email	Entity (getter)	<code>bool get isValidEmail</code>
Guardar en base de datos	DataSource	<code>await box.put(id, model)</code>
Decidir si uso cache o API	Repository	<code>if (isConnected) useRemote()</code>
Calcular impuestos	UseCase	<code>CalculateTaxUseCase</code>
Mostrar indicador de carga	Cubit (State)	<code>UserLoading()</code>
Navegar a otra pantalla	UI (Widget)	<code>Navigator.push(...)</code>

Preguntas Frecuentes

¿Entity debe extender de Model? NO. Son responsabilidades diferentes.

- Entity = Lógica de negocio pura
- Model = Serialización técnica

¿UseCase debe tener solo un método? Sí, el método `call()`. Cada UseCase hace UNA sola cosa.

¿Puedo llamar a un UseCase desde otro UseCase? NO. Los UseCases son independientes.

¿Repository puede tener lógica de negocio? NO. Solo decide fuente de datos y convierte Model ↔ Entity.

¿Puedo usar el Model en la UI? NO. La UI solo trabaja con Entities.

10. Migración desde Código Espaguete

Estrategia de Migración Gradual

No necesitas reescribir todo de una vez. Migra feature por feature.

Paso 1: Aislar una feature

```
// Antes: Todo mezclado
class UserPage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return FutureBuilder(
      future: http.get(Uri.parse('/api/users')), // × HTTP en UI
      builder: (context, snapshot) { /* ... */ },
    );
  }
}
```

Paso 2: Crear la estructura de carpetas

```
lib/features/user/
├── data/
├── domain/
└── presentation/
```

Paso 3: Mover código gradualmente

1. Mueve los modelos JSON a `data/models`
2. Crea las Entities puras en `domain/entities`
3. Extrae la lógica de UI a un Cubit

Paso 4: Conectar todo con inyección de dependencias

Checklist de Migración

- ☐ Feature seleccionada (empezar por la más simple)
- ☐ Estructura de carpetas creada
- ☐ Entity extraída del modelo anterior
- ☐ Repository interface creada
- ☐ UseCases extraídos
- ☐ Cubit/State creado
- ☐ DataSources implementados
- ☐ Repository implementation completada
- ☐ Inyección de dependencias configurada
- ☐ Tests escritos
- ☐ Feature probada completamente

10: SOLID Explicite con Dart/Flutter

SOLID no es teoría académica. Son 5 reglas que evitan que tu código sea un infierno para mantener.

S — Single Responsibility Principle

"Una clase, una razón para cambiar"

```
// ✗ VIOLACIÓN: UserManager hace todo
class UserManager {
  void createUser(String name, String email) { /* crear */ }
  void sendWelcomeEmail(String email) { /* email */ }
  void saveToDatabase(User user) { /* guardar */ }
}

// ✔ CORRECTO: Cada clase tiene UNA responsabilidad
class CreateUserUseCase {
  final UserRepository _repo;
  final EmailService _email;

  CreateUserUseCase(this._repo, this._email);

  Future<void> call(String name, String email) async {
    final user = User(name: name, email: email);
    await _repo.save(user);
    await _email.sendWelcome(email);
  }
}
```

Pregunta clave: "Si esto cambia, ¿por qué razón cambió?"

O — Open/Closed Principle

"Abierto para extender, cerrado para modificar"

```
// × VIOLACIÓN: Para agregar un tipo nuevo, modificas el código
class NotificationSender {
    void send(Notification notif) {
        if (notif.type == 'email') { /* email */ }
        else if (notif.type == 'push') { /* push */ }
        // Agregar SMS = modificar esta clase
    }
}

// ✔ CORRECTO: Nuevos tipos sin modificar código existente
abstract class NotificationSender {
    void send(Notification notif);
}

class EmailSender implements NotificationSender {
    @override
    void send(Notification notif) { /* email */ }
}

class PushSender implements NotificationSender {
    @override
    void send(Notification notif) { /* push */ }
}

// Agregar SMS = crear nueva clase, NO modificar existente
class SmsSender implements NotificationSender {
    @override
    void send(Notification notif) { /* sms */ }
}
```

Pregunta clave: "¿Puedo agregar funcionalidad sin tocar código existente?"

L — Liskov Substitution Principle

"Los subtipos deben ser sustituibles por sus padres"

```
// × VIOLACIÓN: Quadrilateral y Square no son intercambiables
abstract class Shape {
    int get area;
}

class Rectangle implements Shape {
    int width, height;
    Rectangle(this.width, this.height);
    @override
    int get area => width * height;
}

class Square implements Rectangle {
    // Square fuerza width == height, rompiendo expectativas
}

// ✅ CORRECTO: Herencia que no rompe comportamiento
abstract class Shape {
    int get area;
}

class Rectangle implements Shape {
    int width, height;
    Rectangle(this.width, this.height);
    @override
    int get area => width * height;
}

class Square implements Shape {
    int side;
    Square(this.side);
    @override
    int get area => side * side;
}
```

Pregunta clave: "¿Puedo usar el subtipo donde espero el padre sin sorpresas?"

I — Interface Segregation Principle

"Ningún cliente debería depender de métodos que no usa"

```
// × VIOLACIÓN: Interfaz gigante obliga a implementar todo
abstract class UserRepository {
    Future<User> getUser(String id);
    Future<void> saveUser(User user);
    Future<void> deleteUser(String id);
    Future<List<User>> searchUsers(String query);
    Future<void> sendInvitation(String userId);
}

// ✔ CORRECTO: Interfaces pequeñas y específicas
abstract class UserReader {
    Future<User> getUser(String id);
}

abstract class UserWriter {
    Future<void> saveUser(User user);
    Future<void> deleteUser(String id);
}

abstract class UserSearcher {
    Future<List<User>> searchUsers(String query);
}

// Una implementación puede satisfacer múltiples interfaces
class SupabaseUserRepository implements UserReader, UserWriter, UserSearcher {
    @override
    Future<User> getUser(String id) async { /* ... */ }
    @override
    Future<void> saveUser(User user) async { /* ... */ }
    @override
    Future<void> deleteUser(String id) async { /* ... */ }
    @override
    Future<List<User>> searchUsers(String query) async { /* ... */ }
}
```

Pregunta clave: "¿Este cliente usa TODOS los métodos de esta interfaz?"

D — Dependency Inversion Principle

"Depende de abstracciones, no de concreciones"

```
// × VIOLACIÓN: Clase depende de implementación concreta
class UserRepository {
  final SupabaseClient _supabase = SupabaseClient('url', 'key');
  // Hardcoded dependency = imposible de testear
}

// ✅ CORRECTO: Depende de abstracción
abstract class UserRepository {
  Future<User> getUser(String id);
}

class SupabaseUserRepository implements UserRepository {
  final SupabaseClient _supabase;
  SupabaseUserRepository(this._supabase); // Inyectado

  @override
  Future<User> getUser(String id) async { /* ... */ }
}

// Para testing:
class MockUserRepository implements UserRepository {
  @override
  Future<User> getUser(String id) async => User(id: id, name: 'Test');
}

// Uso con DI (GetIt):
getIt.registerLazySingleton<UserRepository>(
  () => SupabaseUserRepository(getIt<SupabaseClient>()),
);
```

Pregunta clave: "¿Puedo cambiar la implementación sin modificar la clase que la usa?"

Resumen SOLID en Dart

Principio	Regla	Ejemplo Dart
S	Una responsabilidad	UseCase = una acción
O	Extender sin modificar	Interface + implementaciones
L	Sustituibilidad	Subtipos no rompen comportamiento
I	Interfaces pequeñas	<code>UserReader</code> , <code>UserWriter</code> separados
D	Depender de abstracciones	Repository interface + DI

Test de SOLID

Para cada clase, pregunta:

- ☐ S: ¿Esta clase tiene solo una razón para cambiar?
- ☐ O: ¿Puedo agregar funcionalidad sin modificar esta clase?
- ☐ L: ¿Puedo reemplazar esta clase por sus subtipos sin bugs?
- ☐ I: ¿Esta interfaz tiene solo métodos que los clientes usan?
- ☐ D: ¿Depende de abstracciones (interfaces) o de concreciones (clases)?

Siguiente: 11-anti-patrones-clean-architecture.md

11: Anti-patrones en Clean Architecture

Errores comunes que convierten Clean Architecture en "Messy Architecture". Conócelos para detectarlos a tiempo.

Anti-patrón 1: La God Class

Síntoma: Una clase que hace absolutamente todo.

```
// × GOD CLASS
class AppManager {
    final SupabaseClient _supabase;

    // Autenticación
    Future<User> signIn(String email, String pass) async { /* ... */ }
    Future<void> signOut() async { /* ... */ }

    // Posts
    Future<List<Post>> getPosts() async { /* ... */ }
    Future<void> createPost(Post post) async { /* ... */ }

    // Notificaciones
    Future<void> sendNotification(String userId) async { /* ... */ }

    // Utilidades
    String formatDate(DateTime date) => /* ... */;
    bool isValidEmail(String email) => /* ... */;
}
```

Solución: Separar por responsabilidad → UseCases, Services, Utils.

Anti-patrón 2: La Dependencia Invertida (en mal sentido)

Síntoma: Domain depende de Data o Presentation.

```
× Domain importa de Data
× Presentation importa de Data directamente
× UseCase importa de SupabaseClient
```

Regla de oro: Las flechas de dependencia SOLO apuntan hacia adentro (hacia Domain).

```
Presentation → Domain ← Data
                ↑
            (Domain no depende de nadie)
```

Anti-patrón 3: El Repository de Papel

Síntoma: Repository interface no tiene sentido, solo pasa datos sin lógica.

```
// × REPOSITORY INÚTIL
abstract class UserRepository {
    Future<User> getUser(String id);
}

class UserRepositoryImpl implements UserRepository {
    final SupabaseClient _supabase;

    @override
    Future<User> getUser(String id) async {
        // Solo passthrough, cero lógica
        final data = await _supabase.from('users').select().eq('id', id).single();
        return User.fromJson(data);
    }
}
```

Cuándo está bien: Si la lógica es solo mapeo de datos, está OK. **Cuándo es problema:** Si NO hay lógica de negocio en UseCases → el UseCase también es inútil.

Anti-patrón 4: El Estado Monolítico

Síntoma: Un solo State para toda la pantalla.

```
// × STATE GIGANTE
class HomeState {
    final List<Post> posts;
    final List<Comment> comments;
    final User? currentUser;
    final bool isLoadingPosts;
    final bool isLoadingComments;
    final String? errorPosts;
    final String? errorComments;
    final int selectedTab;
    final ScrollController scrollController;
    // ... 20 campos más
}
```

Solución: Separar por dominio o usar múltiples BLoCs/Cubits.

```
// ✓ STATES SEPARADOS
class PostsState { /* solo posts */ }
class CommentsState { /* solo comments */ }
class AuthState { /* solo auth */ }
```

Anti-patrón 5: El Usecase Castrado

Síntoma: UseCase que solo hace una llamada sin lógica.

```
// × USECASE INÚTIL
class GetUserUseCase {
    final UserRepository _repo;
    GetUserUseCase(this._repo);

    Future<User> call(String id) async => _repo.getUser(id);
}
```

Solución: Si el UseCase no tiene lógica, únelo al Repository o agrega lógica.

```
// ✅ USECASE CON LÓGICA
class GetUserUseCase {
    final UserRepository _repo;
    final CacheService _cache;

    GetUserUseCase(this._repo, this._cache);

    Future<User> call(String id) async {
        // 1. Check cache
        final cached = await _cache.get<User>('user_$id');
        if (cached != null) return cached;

        // 2. Fetch from repo
        final user = await _repo.getUser(id);

        // 3. Cache for next time
        await _cache.set('user_$id', user);

        return user;
    }
}
```

Anti-patrón 6: El Map hell

Síntoma: `Map<String, dynamic>` por todos lados.

```
// ✗ MAPS POR TODOS LADOS
class UserRepository {
    Future<Map<String, dynamic>> getUser(String id) async {
        return await _supabase.from('users').select().eq('id', id).single();
    }
}

// En el UI:
final name = user['name'] as String; // Crash si falta
final email = user['email'] as String; // No hay compile-time safety
```

Solución: Modelos tipados siempre.

```
// ✅ MODELOS TIPADOS
class User {
    final String id;
    final String name;
    final String email;

    User({required this.id, required this.name, required this.email});

    factory User.fromJson(Map<String, dynamic> json) => User(
        id: json['id'] as String,
        name: json['name'] as String,
        email: json['email'] as String,
    );

    Map<String, dynamic> toJson() => {'id': id, 'name': name, 'email': email};
}
```

Anti-patrón 7: La Herencia Infinita

Síntoma: 5 niveles de herencia donde 3 son abstractos innecesarios.

```

AbstractBaseRepository
├── AbstractRepositoryMixin
│   ├── BaseRepositoryImpl
│   │   ├── UserRepositoryImpl
│   │   └── SupabaseUserRepositoryImpl

```

Solución: Composición sobre herencia. Un nivel de abstracción basta.

```

// ✅ COMPOSICIÓN
class UserRepository {
    final SupabaseDataSource _dataSource;
    final CacheService _cache;

    UserRepository(this._dataSource, this._cache);
    // Usa sus dependencias, no hereda de nada
}

```

Anti-patrón 8: El Callback Hell

Síntoma: Anidamiento de callbacks que hace el código ilegible.

```

// ✖ CALLBACK HELL
_fetchUser((user) {
  _fetchPosts(user.id, (posts) {
    _fetchComments(posts[0].id, (comments) {
      setState(() { /* ... */ });
    });
  });
});

```

Solución: async/await + manejo de errores.

```

// ✅ ASYNC/AWAIT
Future<void> loadData() async {
  try {
    final user = await _fetchUser();
    final posts = await _fetchPosts(user.id);
    final comments = await _fetchComments(posts[0].id);
    emit state;
  } catch (e) {
    emit error;
  }
}

```

Checklist Anti-patrones

- ☐ ¿Algún archivo tiene >300 líneas? (God Class)
- ☐ ¿Domain importa de Data? (Dependencia invertida)
- ☐ ¿Algún UseCase solo hace passthrough? (Usecase castrado)
- ☐ ¿Un solo State maneja 10+ campos? (Estado monolítico)
- ☐ ¿Map<String, dynamic> circula por la app? (Map hell)
- ☐ ¿Más de 3 niveles de herencia? (Herencia infinita)
- ☐ ¿Más de 3 niveles de callbacks? (Callback hell)

Siguiente: [12-cuando-no-usar-clean-architecture.md](#)

12: Cuándo NO Usar Clean Architecture

Clean Architecture no es la respuesta a todo. Usarla donde no corresponde es over-engineering que te frena.

La regla de oro

"No uses arquitectura para un proyecto que no la necesita."

Cuándo SÍ usar Clean Architecture

Situación	¿Por qué?
App con 10+ features	Necesitas separación para no enloquecer
Equipo de 3+ devs	Cada uno trabaja en una capa sin chocar
App con 2+ años de vida	Mantenibilidad a largo plazo
Múltiples fuentes de datos	Repo abstracta facilita cambiar Supabase por API
Testing exhaustivo	Capas facilitan mocking
App con dominio complejo	Business logic compleja necesita estructura

Cuándo NO usar Clean Architecture

Situación	¿Por qué no?	¿Qué usar?
Prototipo / MVP	Velocidad > arquitectura	Código directo
App de una pantalla	No hay separación que valga	Cubit simple + API directa
Script / Tool	No es una app Flutter	Dart script
Hackathon (24h)	No hay tiempo	Lo que funcione
App solo UI (sin lógica)	No hay domain layer	Widgets directos
POC interna	Es desechable	Código simple

Ejemplo: Prototipo sin Clean Architecture

```
// ✗ OVER-ENGINEERING para un prototipo
// 5 archivos, 3 capas, 2 abstracciones... para mostrar 1 lista

// lib/domain/entities/post.dart
// lib/domain/repositories/post_repository.dart
// lib/domain/usecases/get_posts.dart
// lib/data/models/post_model.dart
// lib/data/datasources/post_remote_datasource.dart
// lib/data/repositories/post_repository_impl.dart
// lib/presentation/cubit/posts_cubit.dart
// lib/presentation/cubit/posts_state.dart
// lib/presentation/pages/home_page.dart
// = 9 archivos para una lista de posts

// ✅ PROTOTIPO RÁPIDO (mismo resultado, 1 archivo)
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';

class Post {
  final int id;
  final String title;
  Post({required this.id, required this.title});
  factory Post.fromJson(Map<String, dynamic> json) =>
    Post(id: json['id'], title: json['title']);
}

class HomePage extends StatefulWidget {
  @override
  State<HomePage> createState() => _HomePageState();
}

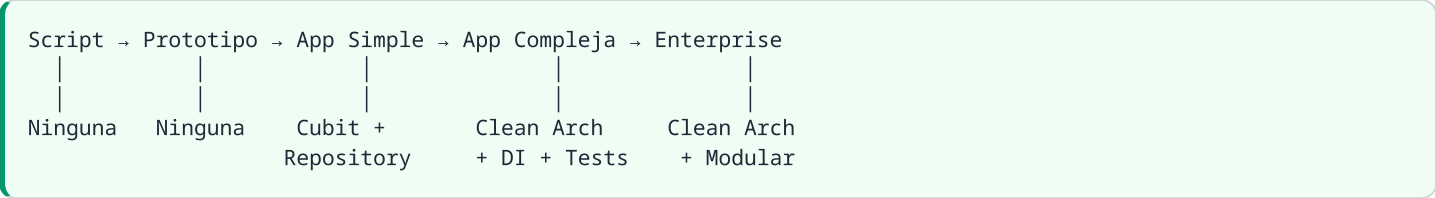
class _HomePageState extends State<HomePage> {
  List<Post> posts = [];
  bool loading = true;

  @override
  void initState() {
    super.initState();
    _load();
  }

  Future<void> _load() async {
    final res = await http.get(Uri.parse('https://jsonplaceholder.typicode.com/posts'));
    setState(() {
      posts = (jsonDecode(res.body) as List).map((j) => Post.fromJson(j)).toList();
      loading = false;
    });
  }

  @override
  Widget build(BuildContext context) {
    if (loading) return CircularProgressIndicator();
    return ListView.builder(
      itemCount: posts.length,
      itemBuilder: (_, i) => ListTile(title: Text(posts[i].title)),
    );
  }
}
```

El Spectrum de Complejidad



Señales de que estás over-engineering

Señal	Qué hacer
Más archivos que features	Reducir abstracciones
UseCases que solo pasan datos	Eliminar UseCases innecesarios
Interfaces que solo tienen 1 implementación	Eliminar la interfaz
Más de 3 niveles de abstracción	Aplanar la estructura
Tardas 30 min en agregar una feature	La arquitectura te frena

La regla práctica

¿Tiene más de 3 features?

→ Clean Architecture vale la pena

¿Tiene menos de 3 features?

→ Cubit + Repository simple

¿Es desechable?

→ Lo que funcione

¿Tiene testing?

→ Clean Architecture facilita

¿No tiene testing?

→ Clean Architecture no ayuda tanto

Ejemplo: App real que evoluciona

Fase 1 (semana 1-2): Prototipo

→ 1 archivo, todo junto

→ Solo para validar la idea

Fase 2 (semana 3-4): MVP

→ Separar en 2-3 archivos

→ Agregar Cubit

→ Empezar a testear

Fase 3 (mes 2+): Producción

→ Clean Architecture completa

→ DI, Tests, States formales

→ Deploy a users reales

Conclusión

Clean Architecture es una herramienta, no un dogma. Úsala cuando el proyecto lo necesite. No la uses porque "es la correcta".

La mejor arquitectura es la que se adapta a las necesidades reales del proyecto.

Volver al índice: [README.md](#)